

Deep Convolutional Architectures for Automated Food Recognition and Nutritional Intelligence: A Production-Grade Framework

The integration of computational vision into dietary management represents a transformative shift in how individuals interact with nutritional data. The process of translating a two-dimensional image into a granular report of macronutrients and micronutrients necessitates a complex, multi-layered architecture that bridges the gap between raw pixel intensity and biological relevance. This report delineates the technical requirements, architectural considerations, and operational strategies required to build a food recognition system using Convolutional Neural Networks (CNNs), the Food-101 dataset, and the FatSecret Platform API.

The Architectural Foundation of Convolutional Neural Networks

The evolution of food image recognition has transitioned from traditional methods based on manual feature extraction—such as Histogram of Oriented Gradients (HOG) or Support Vector Machines (SVM)—to deep learning paradigms where the model autonomously discovers discriminative visual patterns.¹ At the core of this transition is the Convolutional Neural Network (CNN), a specialized class of deep neural networks designed to process data with a known grid-like topology, such as the pixel arrays of a food image.³

The fundamental operation of a CNN is the convolution, a mathematical process that involves sliding a kernel (a small matrix of weights) across the input image to produce feature maps.

Mathematically, for a two-dimensional image I and a filter K , the output feature map S at position (i, j) is computed as:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

This operation allows the network to achieve local connectivity and parameter sharing, ensuring that the model can recognize features like the texture of a crust or the color of a sauce regardless of their location in the image.³ Hierarchical feature extraction is the hallmark of this architecture; early layers detect primitive edges and corners, middle layers synthesize

these into textures and shapes, and deep layers identify complex object parts such as food components.⁵

Feature Transformation and Non-Linearity

Following the convolutional layers, the network applies non-linear activation functions to introduce the complexity required to model non-linear relationships in visual data. The Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$, is the industry standard due to its ability to mitigate the vanishing gradient problem and accelerate convergence.³ To reduce the spatial dimensions of the feature maps and provide a degree of translation invariance, pooling layers—typically Max Pooling—are utilized. Max pooling takes the maximum value within a predefined window, effectively down-sampling the image while preserving the most prominent visual signals.³

CNN Layer Type	Operational Mechanism	Influence on Food Recognition Accuracy
Convolutional	Weight-sharing filters	Captures oil glazes, garnishes, and surface textures. ³
ReLU Activation	Thresholded identity	Enables deep networks to learn non-linear food patterns. ⁶
Max Pooling	Region-based down-sampling	Ensures robustness against image rotation and scaling. ⁷
Global Average Pooling	Spatial aggregation	Reduces parameter count to prevent overfitting on small datasets. ⁸
Dense (Fully Connected)	All-to-all mapping	Finalizes the mapping of features to the 101 food classes. ⁸

The final classification is performed by a Softmax layer, which converts the raw outputs (logits) of the last fully connected layer into a probability distribution over the 101 categories in the dataset.⁶ The objective during the training phase is to minimize the Categorical Cross-Entropy loss, which quantifies the discrepancy between the predicted distribution and the actual

ground-truth label.³

Leveraging the Food-101 Dataset for Discriminative Learning

The choice of dataset is as critical as the choice of architecture. The Food-101 dataset, originally introduced by Bossard et al. in 2014, remains a cornerstone for benchmarking food recognition models.² It comprises 101,000 images distributed across 101 categories, with 1,000 images per class.⁶ A defining characteristic of this dataset is that the training images were purposefully left uncleaned, containing a non-trivial amount of noise in the form of intense colors, motion blur, and incorrect labels.⁶

The existence of noisy labels in the training set (e.g., an omelette labeled as "french toast") forces the model to learn more robust, discriminative components rather than overfitting to specific, potentially erroneous image characteristics.⁹ However, this also poses a significant challenge for validation and testing, as the performance metrics must reflect the model's ability to generalize to clean, manually reviewed test data, which comprises 250 images per class.⁹

Dataset Preprocessing and Normalization

To prepare the Food-101 images for CNN processing, they must undergo a rigorous preprocessing pipeline. Standard practice involves resizing images to a uniform dimension, such as 224x224 or 256x256 pixels, to ensure consistent input shapes for the convolutional filters.¹ Furthermore, normalization is performed using ImageNet statistics—a mean of $[0.485, 0.456, 0.406]$ and a standard deviation of $[0.229, 0.224, 0.225]$ —to align the input data with the weights of pre-trained models.¹

Data augmentation is a critical strategy to enhance model robustness and prevent overfitting, particularly given the high intra-class variation where the same dish may be presented in diverse plating styles.¹ Effective transformations include random horizontal and vertical flips, rotations up to 15 degrees, and color perturbations such as adjustments to brightness and contrast.¹ These techniques simulate various real-world conditions, such as different lighting environments or camera angles, which are common in user-submitted photos.²

Advanced Training Strategies and Transfer Learning

Training a deep CNN from scratch requires enormous computational resources and a vast amount of labeled data. For a developer building a food recognition application, transfer learning is the most efficient approach.¹ This involves taking a model that has been pre-trained on a massive dataset like ImageNet and fine-tuning it for the 101 food categories.¹

Architectures like ResNet-50 are frequently chosen due to their "skip connections," which allow the gradient to flow through the network more effectively, enabling the training of deeper

models without the risk of vanishing gradients.¹ Alternatively, for mobile deployment where latency is a concern, MobileNetV2 uses depth-wise separable convolutions to provide high accuracy with a significantly lower parameter count.²

Model Architecture	Computational Efficiency	Accuracy on Food-101	Recommended Use Case
ResNet-18	High	Moderate (~91% on 5 classes) ¹	Real-time desktop applications.
ResNet-50	Moderate	Moderate to High (~55% on 101 classes) ¹	Server-side API processing.
MobileNetV2	Very High	High ²	On-device mobile recognition.
VGG-19	Low	High ²	Research-focused high-precision tasks.

The transfer learning process typically begins by freezing the weights of the convolutional base and training a newly added classification head. Once the head has converged, the developer may "unfreeze" the final few blocks of the convolutional base and retrain them with a very low learning rate—a technique known as discriminative fine-tuning—to adapt the high-level features to the specific nuances of food images.⁹

Statistical Validation and Testing Methodologies

The integrity of a food recognition model is determined by how it handles data it has never encountered. To achieve this, a developer must meticulously divide the original dataset into three distinct subsets: training, validation, and testing.³

The Three-Way Dataset Split

The training set is the primary source of knowledge, used to update the network's weights through backpropagation. However, testing the model on the same data it was trained on provides no evidence of its ability to generalize; a model that achieves 99% accuracy on the training set but 50% on unseen data is suffering from overfitting.³

The validation set serves as an intermediary evaluation tool during the training process. It acts

as a "reality check" to tune hyperparameters such as the learning rate, dropout rate, and batch size.³ By monitoring the validation loss, developers can implement "early stopping," where training is halted when the validation performance begins to degrade, even if the training loss is still decreasing.⁵

The test set remains strictly isolated until the final model version is chosen. It provides an unbiased estimate of the model's performance in a real-world scenario.³ In the context of Food-101, the provided test images are manually reviewed to ensure they are free from the noise present in the training set, providing a high-fidelity benchmark for final evaluation.⁹

Stratification and Cross-Validation

To ensure that each of the 101 classes is equally represented, stratification must be applied during the split.¹⁷ This ensures that the model is not biased toward classes that might have more visual variety or better quality images. In scenarios where data is limited, 5-fold cross-validation is often employed to maximize the utility of the dataset, although this is computationally expensive for deep CNNs on the full Food-101 dataset.¹

Open-Set Recognition: Handling the "Unknown"

A critical requirement for a production-ready application is the ability to recognize when an image does not belong to any of the 101 trained categories. Traditional "closed-set" classifiers will confidently assign a label like "apple pie" to an image of a dog or a car because they are forced to choose from the existing classes.¹⁸

This challenge is addressed through Open-Set Recognition (OSR) and Out-of-Distribution (OOD) detection.¹⁸ One common approach is thresholding the Softmax output; if the highest probability assigned to any class falls below a certain confidence level (e.g., 0.8), the system rejects the input as "Unknown".²⁰

OpenMax and Extreme Value Theory

A more sophisticated technique is the OpenMax algorithm, which recalibrates the final layer scores using the Weibull distribution.²⁶ OpenMax estimates the probability that a sample is an outlier by measuring its distance from the "centroids" of the known classes in the feature space.²⁷ If the sample is too far from any known class center, the model increments a $(K + 1)$ -th category representing "unknown".²⁷

Another robust method is Classification-Reconstruction learning for Open-Set Recognition (CROSR). In this framework, the network is trained to classify the image while simultaneously attempting to reconstruct it.¹⁹ Known classes will have low reconstruction errors because the network has learned their latent representations, whereas unknown items will result in high reconstruction errors, providing a clear signal for rejection.¹⁹

The Continuous Improvement Loop: Active Learning

Model performance in the lab rarely translates perfectly to the real world, where lighting conditions, camera quality, and regional food variations introduce constant "drift".²⁹ To ensure the model improves over time, developers implement an Active Learning (AL) pipeline.⁵

Active Learning is an iterative process where the model identifies the most informative unlabeled data points—those it is most uncertain about—and queries a human "oracle" to provide labels.³¹ This is significantly more efficient than labeling all new data, as it focuses human effort on the samples that will most likely improve the model's accuracy.³¹

Active Learning Strategy	Selection Mechanism	Benefit for Food Recognition
Uncertainty Sampling	Samples with lowest top-1 probability ³¹	Targets visually ambiguous dishes (e.g., soups vs. stews). ⁴⁰
Entropy Sampling	Samples with highest prediction entropy ³¹	Identifies items that confuse multiple classes. ³¹
Query by Committee	Disagreement among multiple models ³³	Reduces bias by checking predictions against different architectures. ³³
Diversity Sampling	Representative samples of the data distribution ⁵	Ensures the model learns rare cuisines or new products. ³⁵

By integrating a "feedback loop," the model can be retrained periodically with these new, high-value labels, ensuring that it adapts to emerging food trends and user-specific dietary habits.⁵

Architectural Process of Food Recognition and Nutrient Extraction

The journey from a user's camera to the display of nutritional details involves a complex interplay of mobile frontends, machine learning backends, and third-party APIs. The current state of the art emphasizes the need for asynchronous operations to mitigate high

latency—often cited as a primary driver of poor user retention in health-tech apps.⁴³

The Prediction and Enrichment Workflow

The technical process can be visualized as a sequential flow of data, though in a production environment, many of these steps must happen in parallel to ensure a responsive UI.⁴³

1. **Image Submission:** The user captures a meal image on the PlanningScreen of the mobile application.⁴³
2. **Preprocessing & Inference:** The image is sent to a Python-based backend (e.g., FastAPI or Flask) where it is normalized and passed through the CNN.¹
3. **Label Identification:** The model identifies the food item (e.g., "Pad Thai") and returns a list of candidate labels with confidence scores.¹
4. **FatSecret Search:** The system uses the `foods.search.v3` method to match the predicted label against the FatSecret database, which contains over 2.3 million unique items.⁴⁹
5. **Nutrient Extraction:** Once a `food_id` is confirmed, the system calls `food.get.v4` to retrieve detailed nutritional information, including calories, protein, carbohydrates, and fat.⁴⁷
6. **State Update:** The nutritional data is sent back to the frontend, updating the user's daily log and calorie goals.⁸

Addressing Latency and Integration Challenges

A significant bottleneck in this workflow is the "fetch waterfall," where the application awaits the completion of multiple external API calls before rendering the UI.⁴³ Research indicates that individual FatSecret API calls can take between 200ms and 800ms.⁴³ For a meal containing multiple items, these delays aggregate, leading to a perceived lack of responsiveness.

To mitigate this, developers should implement asynchronous image enrichment and caching.⁴³ By using parallel network requests and local caching (compliant with FatSecret's 24-hour storage limit), the perceived latency can be reduced significantly.⁴³

System Process Diagram Description

The operational logic of the food recognition function is structured as a closed-loop system, ensuring that data flows seamlessly from the user to the database and back.

I. User Interaction Layer (Frontend) The process begins when the user interacts with the camera module of the React Native application. The application captures the image and performs an initial compression to reduce the payload size before transmitting the image via an HTTP POST request to the backend API.⁴³

II. Recognition Engine (Python CNN) The backend receives the raw image and passes it into a preprocessing queue. Here, the image is resized to 224x224 and normalized using standard ImageNet values. The CNN (e.g., ResNet-50) performs a forward pass, outputting a vector of

probabilities across 101 food classes. An OOD detector monitors the Softmax output; if the maximum probability is below 0.75, the system prompts the user to "Verify Category" or select from a list of top-5 candidates.⁸

III. Nutritional Intelligence Layer (FatSecret API Proxy) The confirmed food label is then passed to the FatSecret Integration Service. This service first checks a local Redis cache to see if the nutritional data for that specific item has been fetched in the last 24 hours. If not, it initiates a `foods.search.v3` call. The service selects the most relevant `food_id` (favoring generic items for raw produce or specific brands if detected) and then executes a `food.get.v4` request to retrieve the full macronutrient profile.⁴⁹

IV. Feedback and Persistence Layer The resulting nutritional JSON is returned to the mobile app, where it is mapped to the user's meal slots (Breakfast, Lunch, Dinner). Simultaneously, the image and the final confirmed label are stored in an "Active Learning Pool." This pool is periodically reviewed by human annotators to identify misclassifications, which are then used to retrain the model in the next development cycle.⁵

Strategic Development Backlog and Implementation Phases

The successful deployment of a high-accuracy food recognition model requires a phased roadmap that balances research, development, and operational stability.

Phase 1: Experimental Validation and Data Engineering

The initial phase focuses on establishing a robust data pipeline and a baseline model.

- **Data Pipeline Setup:** Implement scripts to automate the download and organization of the Food-101 dataset, ensuring a clear directory structure for training and testing.¹
- **Exploratory Analysis:** Analyze the Food-101 metadata to identify class imbalances and the prevalence of noisy labels, which will inform the choice of loss functions like Label Smoothing.¹
- **Stratified Splitting:** Develop the splitting logic to create a three-way partition (Train 75%, Val 15%, Test 10%), ensuring every food category is represented in each set.⁵
- **Baseline Training:** Train an initial ResNet-18 model on a 5-class subset of Food-101 to validate the preprocessing and training logic before scaling to the full dataset.¹

Phase 2: Model Optimization and Open-Set Integration

The second phase scales the model to the full dataset and implements safety features for real-world use.

- **Full-Scale Training:** Transition to a ResNet-50 or MobileNetV2 architecture, utilizing transfer learning from ImageNet. Implement discriminative fine-tuning where the convolutional base is frozen initially and then selectively unfrozen.¹

- **Hyperparameter Tuning:** Use the validation set to optimize the learning rate, weight decay, and dropout settings to maximize top-1 and top-5 accuracy.¹
- **OSR Implementation:** Integrate an OpenMax layer or a Softmax thresholding mechanism to detect and reject non-food images or dishes outside the Food-101 distribution.²⁰
- **Performance Benchmarking:** Evaluate the final model against the clean Food-101 test set, reporting precision, recall, and F1-score for each category.¹

Phase 3: Integration and API Development

Phase three bridges the model with the rest of the application ecosystem.

- **Model Serving API:** Develop a Python-based REST API using FastAPI. Package the model and its dependencies into a Docker container to ensure consistent performance across deployment environments.¹
- **FatSecret Integration:** Implement the OAuth 2.0 authentication flow and develop a service that translates CNN predictions into FatSecret search queries. Optimize the mapping logic to handle pluralization and common food aliases.⁵³
- **Latency Optimization:** Implement asynchronous request handling and a 24-hour caching layer for nutritional data to minimize frontend wait times.⁴³

Phase 4: Productionization and Monitoring

The fourth phase focuses on reliability, scaling, and operational visibility.

- **CI/CD/CT Pipeline:** Setup automated retraining (Continuous Training) pipelines where the model is automatically retrained when significant data drift is detected or when a new batch of labeled data becomes available.²⁹
- **Monitoring & Observability:** Implement a Monitoring Layer to track production metrics such as API response times, model confidence levels, and user correction rates.²⁹
- **Drift Detection:** Deploy statistical monitors (e.g., KL divergence or PSI) to identify when the distribution of user-submitted images begins to shift away from the training data, signaling the need for an update.²⁹

Phase 5: Longitudinal Improvement and Active Learning

The final phase establishes the long-term "flywheel" that ensures the model remains a market leader.

- **Active Learning Loop:** Implement an automated system to flag high-entropy or low-confidence samples and route them to an internal labeling interface.³¹
- **User-Centric Feedback:** Allow users to provide "on-the-fly" corrections to the model's predictions, which are then vetted by experts and added to the training pool.⁵
- **Feature Expansion:** Explore multi-task learning to detect ingredients and estimate portion sizes from images, providing even more granular nutritional data.²

Conclusion: Orchestrating a Robust Nutritional Ecosystem

The creation of a food recognition function is not a singular event but a continuous engineering journey. By leveraging the hierarchical feature learning of CNNs and the established benchmark of the Food-101 dataset, a developer can build a foundation of high-precision image classification. However, the true value of the application lies in its ability to handle the "unknown" and improve through every user interaction.

The integration of Open-Set Recognition ensures that the model remains reliable even when faced with unfamiliar foods, while an Active Learning framework transforms user submissions into a powerful engine for continuous improvement. When coupled with an optimized FatSecret API orchestration layer that prioritizes low latency and asynchronous data enrichment, the result is a seamless, intelligent, and deeply personalized meal planning experience. The future of this application lies in its ability to adapt to the evolving global food landscape, ensuring that every photo taken brings the user closer to their health and wellness goals.

Works cited

1. verynewusername/food-101: Food-101: A project exploring ... - GitHub, accessed April 26, 2026, <https://github.com/verynewusername/food-101>
2. Deep Learning in Food Image Recognition: A Comprehensive Review - MDPI, accessed April 26, 2026, <https://www.mdpi.com/2076-3417/15/14/7626>
3. Intro to CNNs, <https://drive.google.com/open?id=16Z2fnBl2azfGxZ8lnHiFvRb1OLZVQTLQpy8GzZR5YEg>
4. Food Recognition Model with Deep Learning Techniques [Updated] - Labellerr, accessed April 26, 2026, <https://www.labellerr.com/blog/food-recognition-and-classification-using-deep-learning/>
5. How to Build an ML Data Pipeline for Computer Vision [Tutorial] - Lightly AI, accessed April 26, 2026, <https://www.lightly.ai/blog/ml-data-pipeline>
6. Food Classification with CNNs - CS230 Deep Learning - Stanford University, accessed April 26, 2026, http://cs230.stanford.edu/projects_fall_2020/reports/55825128.pdf
7. Food Classification from Images Using a Neural Network Based Approach with NVIDIA Volta and Pascal GPUs - MOST Wiedzy, accessed April 26, 2026, https://mostwiedzy.pl/pl/publication/download/1/food-classification-from-images-using-a-neural-network-based-approach-with-nvidia-volta-and-pascal-g_73897.pdf
8. Food-101 Example - Kaggle, accessed April 26, 2026, <https://www.kaggle.com/code/sanadalali/food-101-example>
9. Food-101 - Kaggle, accessed April 26, 2026,

- <https://www.kaggle.com/code/harishvutukuri/food-101>
10. food101 | TensorFlow Datasets, accessed April 26, 2026, <https://www.tensorflow.org/datasets/catalog/food101>
 11. Food101 — Torchvision main documentation, accessed April 26, 2026, <https://docs.pytorch.org/vision/main/generated/torchvision.datasets.Food101.html>
 12. food-101-tiny-experiments - Kaggle, accessed April 26, 2026, <https://www.kaggle.com/code/msarmi9/food-101-tiny-experiments>
 13. Food-101 Classifier using ResNet50 on Colab — Onur Andros Ozbek - Medium, accessed April 26, 2026, https://medium.com/@onur_andros_ozbek/food-101-classifier-using-resnet50-on-colab-f0cc6ac6487
 14. Recognition of food images based on transfer learning and ensemble learning - PMC, accessed April 26, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10798480/>
 15. The Food Recognition Benchmark: Using Deep Learning to Recognize Food in Images - Frontiers, accessed April 26, 2026, <https://www.frontiersin.org/journals/nutrition/articles/10.3389/fnut.2022.875143/full>
 16. Dividing the original dataset | Machine Learning - Google for Developers, accessed April 26, 2026, <https://developers.google.com/machine-learning/crash-course/overfitting/dividing-datasets>
 17. Train/Test/Validation Set Splitting in Sklearn - Data Science Stack Exchange, accessed April 26, 2026, <https://datascience.stackexchange.com/questions/15135/train-test-validation-set-splitting-in-sklearn>
 18. Benchmarking Post-Hoc Unknown-Category Detection in Food Recognition - arXiv, accessed April 26, 2026, <https://arxiv.org/html/2503.18548v1>
 19. Classification-Reconstruction Learning for Open-Set Recognition, accessed April 26, 2026, https://openaccess.thecvf.com/content_CVPR_2019/papers/Yoshihashi_Classification-Reconstruction_Learning_for_Open-Set_Recognition_CVPR_2019_paper.pdf
 20. Are open set classification methods effective on large-scale datasets? - PMC, accessed April 26, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC7473573/>
 21. Can We Ignore Labels in Out of Distribution Detection? - OpenReview, accessed April 26, 2026, <https://openreview.net/forum?id=falBlwUsIH>
 22. Out of Distribution Detection: Knowing When AI Doesn't Know, accessed April 26, 2026, <https://www.sei.cmu.edu/blog/out-of-distribution-detection-knowing-when-ai-doesnt-know/>
 23. What is Out-of-Distribution (OOD) Detection? - Encord, accessed April 26, 2026, <https://encord.com/blog/what-is-out-of-distribution-ood-detection/>
 24. Adapting a Previously Proposed Open-Set Recognition Method for Time-Series Data: A Biometric User Identification Case Study - MDPI, accessed April 26, 2026, <https://www.mdpi.com/2079-9292/14/20/3983>

25. Open-Set Recognition in the Age of Vision-Language Models - arXiv, accessed April 26, 2026, <https://arxiv.org/html/2403.16528v1>
26. ma-xu/Open-Set-Recognition - GitHub, accessed April 26, 2026, <https://github.com/ma-xu/Open-Set-Recognition>
27. nikita-petrashen/openmax - GitHub, accessed April 26, 2026, <https://github.com/nikita-petrashen/openmax>
28. [PDF] Generative OpenMax for Multi-Class Open Set Classification | Semantic Scholar, accessed April 26, 2026, <https://www.semanticscholar.org/paper/Generative-OpenMax-for-Multi-Class-Open-Set-Ge-Demyanov/1643f48027cd18aeaf713bf7b5b18bb91a765503>
29. Automated MLOps Retraining Pipeline for LLM and Recommendation Systems - IJERT, accessed April 26, 2026, <https://www.ijert.org/automated-mlops-retraining-pipeline-for-llm-and-recommendation-systems-ijertv15is040538>
30. Behind the Scenes of AI: How MLOps Keeps Machine Learning Alive | by TheCodeDesk, accessed April 26, 2026, <https://medium.com/@thecodedesk910/behind-the-scenes-of-ai-how-mlops-keeps-machine-learning-alive-3996e155d5d7>
31. Spectroscopy Approaches for Food Safety Applications: Improving Data Efficiency Using Active Learning and Semi-supervised Learning - Frontiers, accessed April 26, 2026, <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2022.863261/full>
32. How Active Learning Can Improve Your Computer Vision Pipeline - DagsHub, accessed April 26, 2026, <https://dagshub.com/blog/how-active-learning-can-improve-your-computer-vision-pipeline/>
33. Machine-Learning/Active Learning in Machine Learning with Python.md at main - GitHub, accessed April 26, 2026, <https://github.com/xbeat/Machine-Learning/blob/main/Active%20Learning%20in%20Machine%20Learning%20with%20Python.md>
34. Active Learning in Machine Learning Explained - Towards Data Science, accessed April 26, 2026, <https://towardsdatascience.com/active-learning-in-machine-learning-explained-777c42bd52fa/>
35. Active Learning Strategies in Data Science: Making Models Smarter, Faster | by DATADRIX, accessed April 26, 2026, <https://medium.com/@datadrix/active-learning-strategies-in-data-science-making-models-smarter-faster-28e561a4be78>
36. Data Science: modAL — Active Learning Framework | by Abhilasha Gulhane | Medium, accessed April 26, 2026, <https://medium.com/@abhilashagulhane111/data-science-modal-active-learning-framework-93763d3f843b>
37. Data-Driven AI in Manufacturing: Casting Defect Identification using Active Learning, accessed April 26, 2026,

- <https://dagshub.com/blog/maximizing-machine-learning-efficiency-with-active-learning/>
38. active-vision: Active Learning Framework for Computer Vision : r/computervision - Reddit, accessed April 26, 2026, https://www.reddit.com/r/computervision/comments/1iiv14n/activevision_active_learning_framework_for/
 39. Active Learning for classification models | SuperAnnotate, accessed April 26, 2026, <https://www.superannotate.com/blog/active-learning-classification-models>
 40. A Comprehensive Survey of Image-Based Food Recognition and Volume Estimation Methods for Dietary Assessment - PMC, accessed April 26, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC8700885/>
 41. [D] Has anyone worked on food recognition models? I'm curious about the accuracy challenges with mixed dishes. : r/MachineLearning - Reddit, accessed April 26, 2026, https://www.reddit.com/r/MachineLearning/comments/1om2ggy/d_has_anyone_worked_on_food_recognition_models_im/
 42. Review Classification using Active Learning - Keras, accessed April 26, 2026, https://keras.io/examples/nlp/active_learning_review_classification/
 43. Meal Recommendation Model Improvement Plan, https://drive.google.com/open?id=1GgqfT5ups2R0DZnkutHjtZyF-vUI_sVMDZCknySw9Ro
 44. 5 Things That Cause High Latency in Your APIs (and How to Fix Them), accessed April 26, 2026, <https://nordicapis.com/5-things-that-cause-high-latency-in-your-apis-and-how-to-fix-them/>
 45. How to Optimize API Calls for Faster Mobile Apps - Digia Studio, accessed April 26, 2026, <https://www.digia.tech/post/optimize-api-calls-faster-mobile-apps>
 46. How to Cut API Latency: Diagnose, Measure, and Optimize Performance - Gravitee, accessed April 26, 2026, <https://www.gravitee.io/blog/cut-api-latency-diagnose-measure-and-optimize>
 47. AI Nutrition API | Food Image Recognition & NLP for Health Apps - fatsecret Platform, accessed April 26, 2026, <https://platform.fatsecret.com/features/image-recognition-and-nlp>
 48. Build a Custom Food Image Recognition Classifier in 5 Steps - Nyckel, accessed April 26, 2026, <https://www.nyckel.com/blog/food-image-recognition/>
 49. The Largest Global Nutrition Database, Recipe and Food API - fatsecret Platform, accessed April 26, 2026, <https://platform.fatsecret.com/platform-api>
 50. Foods: Search | fatsecret Platform API, accessed April 26, 2026, <https://platform.fatsecret.com/docs/v1/foods.search>
 51. Foods: Search | fatsecret Platform API, accessed April 26, 2026, <https://platform.fatsecret.com/docs/v3/foods.search>
 52. Foods: Search | fatsecret Platform API, accessed April 26, 2026, <https://platform.fatsecret.com/docs/v2/foods.search>
 53. fatsecret Platform API Documentation, accessed April 26, 2026, <https://platform.fatsecret.com/docs/guides>

54. FatSecret — Get the Data. An overview of data extraction options... | by Olga | Medium, accessed April 26, 2026, <https://olgarithm.medium.com/fatsecret-get-the-data-b7f3eecf7165>
55. How to Optimize API Response Time to Improve Mobile App Performance During Peak User Traffic - Zigpoll, accessed April 26, 2026, <https://www.zigpoll.com/content/how-can-i-optimize-the-api-response-time-to-improve-the-mobile-apps-performance-during-peak-user-traffic>
56. Storable Data Documentation | fatsecret Platform API, accessed April 26, 2026, <https://platform.fatsecret.com/docs/guides/storable-data>
57. FatSecret API Integration for Nutrition & Food Tracking Apps | SportsFirst, accessed April 26, 2026, <https://www.sportsfirst.net/sportsapi/fatsecret>
58. Caching terms for FatSecretAPI - Google Groups, accessed April 26, 2026, <https://groups.google.com/g/fatsecret-platform-api/c/b4leh4EXqwo>
59. 5 Battle-Tested Strategies to Supercharge Your API Performance - DEV Community, accessed April 26, 2026, https://dev.to/dehemi_fabio/5-battle-tested-strategies-to-supercharge-your-api-performance-22ah
60. food101-classification/split_train_test_folder.py at master - GitHub, accessed April 26, 2026, https://github.com/DucLeTrong/food101-classification/blob/master/split_train_test_folder.py
61. Alcrowd | Food Recognition Challenge, accessed April 26, 2026, <https://www.aicrowd.com/challenges/food-recognition-challenge>
62. What Is MLOps, How to Implement It, Examples - Dysnix, accessed April 26, 2026, <https://dysnix.com/blog/what-is-mlops>
63. FatSecret MCP Server - LobeHub, accessed April 26, 2026, <https://lobehub.com/mcp/lkotoula-fatsecret-mcp>
64. Natural Language Processing | fatsecret Platform API, accessed April 26, 2026, <https://platform.fatsecret.com/docs/v1/natural.language.processing>
65. A Complete Guide to MLOps Architecture in 2026 - Moon Technolabs, accessed April 26, 2026, <https://www.moontechnolabs.com/blog/mlops-architecture/>